

Images at realistic scale: transfer learning

Session 16's digits were 64 pixels. A phone photograph has millions. Ten minutes on why the dense network cannot follow and what a convolutional filter does instead — then the session's real business: taking a network that already learned to see from over a million photographs, and teaching its last layer your two classes with a few hundred images.

Unit 4 · Real data

80 minutes

Lab 16 · change the data

Depth: CV series L1–L5 as reading

[The scale problem](#)[Transfer learning](#)[The code](#)[Limits](#)[Lab 16](#)[Before Friday](#)

PART 1 · 10 MINUTES

Why a dense network on pixels does not scale

1

64INPUTS — SESSION 16
DIGITS**150,528**INPUTS — ONE 224 ×
224 COLOUR IMAGE**150
million**WEIGHTS IN ONE
DENSE LAYER OF 1,000
NEURONS**28**NUMBERS IN ONE 3 × 3
COLOUR FILTER

A 224 × 224 colour image is $224 \times 224 \times 3 = 150,528$ numbers. Connect that to a hidden layer of 1,000 neurons and the first weight matrix alone holds over 150 million numbers — before the network has learned anything, and with no idea that neighbouring pixels belong together or that a cat in the corner is still a cat.

WHAT A CONVOLUTIONAL FILTER DOES ABOUT IT

Slide one small set of weights — $3 \times 3 \times 3$ plus a bias, 28 numbers — across the whole image, computing the same weighted sum at every position. **Weight sharing** makes it cheap; **locality** makes it look at neighbourhoods; and a pattern detected in one place is detected everywhere. Stack such layers and early ones find edges, later ones textures, later still parts of objects.



Convolution is not re-derived today

The Computer Vision series carries that load as assigned reading: L1 Images as Data, L2 Convolution, L3 CNN Architecture, L4 Historical Victories, L5 Transfer Learning. Deck slides 19, 21 and 23 (v2) are the summary.

PART 2 · 20 MINUTES

Transfer learning: borrow the eyes, train the verdict

2

Training a good image network from scratch takes a very large labelled dataset and a lot of compute. A clinic in Grand-Bassam with 400 photographs has neither. But the early and middle layers of a network trained on ImageNet — 1.2 million photographs in 1,000 categories — have learned things that are useful for almost any photograph: edges, textures, shapes, parts.

KEEP WHAT WAS LEARNED ABOUT SEEING; REPLACE ONLY THE FINAL DECISION

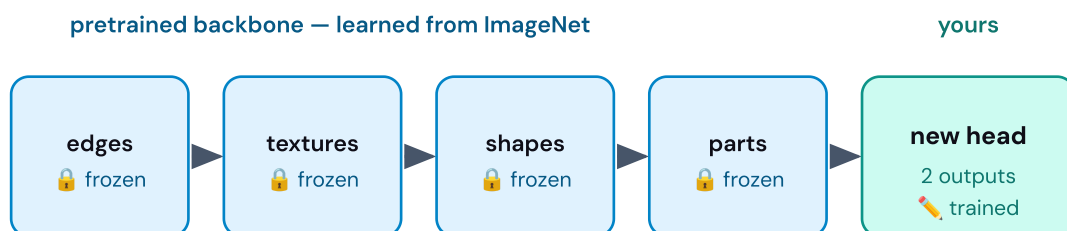


image → features that describe almost any photograph → your two classes

1 Load a pretrained network — Lab 16 uses MobileNetV3–Small, built to be light enough for phones.

- 2 **Freeze** its feature extractor: `requires_grad = False` for every parameter, so backprop leaves them alone.
- 3 **Replace** the last layer — 1,000 ImageNet outputs — with a new one that has two.
- 4 **Train** only what is not frozen, for a few epochs, on your small dataset.

THE SAME MOVE YOU ALREADY KNOW

This is session 16's training loop with most parameters switched off. The backbone turns each image into a feature vector; the new head is a small classifier on those features — close to the neuron from session 11. **You are training a small model on very good features someone else learned.**



Use the backbone's own preprocessing

The pretrained network expects images resized, cropped and normalised exactly as during its original training. `weights.transforms()` supplies that pipeline. Feeding it differently scaled images is session 19's scaling bug, in a new costume.

PART 3 · 15 MINUTES

The code, and what to change

The notebook is provided and works. Your job is the data.

3

the heart of Lab 16

PYTHON

```
model = models.mobilenet_v3_small(weights=models.MobileNet_V3_Small_Weights.DEFAULT)
for p in model.features.parameters():
    p.requires_grad = False  # frozen
model.classifier[-1] = nn.Linear(model.classifier[-1].in_features, 2)  # new head

opt = torch.optim.Adam([p for p in model.parameters() if p.requires_grad], lr=1e-3)
```

Only the classifier layers after the feature extractor are trainable — a few hundred thousand parameters out of about 1.5 million, most of them in the classifier's hidden layer. Three epochs on 400 CIFAR-10 images of cats and dogs typically moves validation accuracy from chance to well above it. Then change `CLASSES`, or point the notebook at your own folder of photographs.



Local data

Forty photographs per class is enough to see transfer learning work: two local crops, two kinds of market produce, two types of road damage. The session-5 grid you sliced by hand is the same object as every one of these images — just larger, and in colour.

PART 4 · 10 MINUTES

When transfer learning works, and when it does not

4

Works well when...	Struggles when...
the new images resemble photographs	the images are very different: X-rays, satellite bands, microscopy
classes differ in visible shape or texture	the difference is subtle and needs expert features
you have tens to hundreds of images per class	you have a handful, or wildly unbalanced classes

When the images are very different, unfreeze the last block or two and train them with a small learning rate — the lab's extension — or look for a backbone pretrained on similar images. And every lesson of unit 4 still applies: a test set touched once, a baseline, and the question of which mistakes cost more.

LAB 16 · 25 MINUTES IN CLASS

Transfer learning on images

5

Lab 16 — `CSC4315_Lab16_Transfer_Learning_0n_Images`, on Colab with a GPU runtime or on `atd_env`. Run it as provided, then with a different pair of classes or your own photographs, and fill the table.

Before Friday

1. **Finish Lab 16.** Unit 4 ends here.
2. **Read** CV series L5 (Transfer Learning) if you have not.

3. **Think about this:** a network needs numbers. How would you turn the sentence "*the clinic needs medicine*" into numbers that keep some of its meaning?



Session 24 — Friday 27 November, 11:30

Unit 5 begins: text as numbers. Tokens, vocabulary, and the idea that meaning can become coordinates.